Frontiers in Computing: Discussion 5

Rento Saijo

2026-10-04

Processors for different kinds of work

- Computing tasks demand different combinations of flexibility, throughput, and efficiency, which explains why systems use several kinds of processors. The central processing unit (CPU) handles varied instructions, decisions, and coordination: it can run an application, respond to a web request, or direct other hardware. It also performs parallel work, especially across multiple cores, but a graphics processing unit (GPU) devotes more of its hardware to applying similar calculations to many data items at once. That arrangement suits the pixels and geometric shapes in a game frame as well as portions of scientific computing and machine learning.
- Further specialization follows the same principle. Google's tensor processing unit (TPU) targets the matrix operations common in suitable machine-learning workloads, whereas a neural processing unit (NPU) can run features such as image enhancement or speech recognition efficiently on a phone. In a data center, a data processing unit (DPU) can offload networking, storage, security, and data movement from application CPUs. A quantum processing unit (QPU) uses qubits and quantum algorithms to investigate particular classes of problems; it is no general replacement for a classical processor. One system can therefore combine several kinds of processing instead of choosing a single winner.

A photograph through the smartphone

- The smartphone gives us a compact example of cooperation. Its system on a chip (SoC) brings CPU cores, a GPU, an image signal processor (ISP), display and media engines, communication hardware, controllers, and sometimes an NPU onto a connected platform. An on-chip network moves information among these blocks, while controllers manage access to memory. Working data resides in dynamic random-access memory (DRAM), and a saved photo resides in flash storage; neither should be confused with a processing unit inside the SoC. This organization lets specialized blocks handle frequent tasks without asking the CPU to perform every calculation itself.
- When we take a photo, light reaches the image sensor, whose measurements are converted into digital values. The resulting data travel into working memory and through the ISP, which can reconstruct color from the sensor's color-filter pattern, reduce noise, and adjust color and tone. Processing occurs as data move through the system; the entire image need not fit in a small on-chip cache. Meanwhile, the CPU runs the camera application and coordinates the sequence. If the phone uses learned image enhancement, an NPU may assist, although the exact processing path varies by device and feature.
- To show the preview, the GPU can combine the camera image with the application's controls, and the display engine sends the composed result to the screen. Saving the picture adds encoding and a write to flash; sharing it invokes the modem and networking path. These stages explain why a smartphone contains many engines even though the user experiences one quick tap. Power management matters too: the phone can assign work to more efficient CPU cores or dedicated accelerators and change operating speed as demand rises or falls, extending battery life without changing the photograph's purpose.

Inside the CPU

- Inside a CPU, transistors act as controlled switches; arranged into logic gates and circuits, they can add values, make comparisons, and hold bits in registers. A running program gives those circuits a sequence of instructions. The CPU fetches an instruction, decodes what operation it requests, obtains operands, executes the operation, and makes the result available for later work. To add two values, for example, the processor may load them into registers, use an arithmetic unit to add them, and retain or store the sum. The control logic decides which units participate, while a program counter tracks the next instruction.
- Fetching data is often as important as arithmetic. Registers are closest to the executing units; small, fast caches keep recently used instructions and data nearby; larger DRAM holds more of the active program at a greater access cost. A cache miss sends the processor farther down this hierarchy. In our photo example, software may repeatedly inspect nearby image values, so reusing data already in cache can save trips to memory. The hierarchy also explains why a fast calculation does not guarantee a fast program: if the required inputs arrive slowly, arithmetic hardware waits.
- Modern CPUs overlap stages of several instructions in a pipeline, much as different stations work on successive items at once. A branch prediction lets the pipeline continue before a conditional choice is resolved; a wrong guess forces work on the incorrect path to be discarded. Separate cores execute independent instruction streams, and vector instructions apply one operation to several values within a core. Simultaneous multithreading lets one core keep instructions from more than one software thread in flight, improving use of shared execution resources when possible. Those logical threads do not create additional physical cores, and the gain depends on the program.

Matching CPUs, GPUs, and TPUs to work

- A web request shows the CPU's strength: read the request, check identity, look up information, apply rules, and choose a response. Each step can lead to another path. In contrast, rendering many image elements or repeating the same numerical operation over a large data set exposes parallel work that a GPU can process at high throughput. Both processors can often perform the task, but their organization changes how efficiently they do it. The CPU also remains valuable when it prepares data, launches GPU work, and handles decisions around that work.
- Machine learning adds another comparison. A matrix is a grid of numbers, and multiplying matrices repeats many multiply-and-add operations. A color image can be represented as a tensor with height, width, and color channels; a model may transform such numerical arrays through many layers. GPUs handle a broad range of this parallel arithmetic, while Google's TPUs devote specialized hardware to matrix-heavy neural-network computations. A TPU can be advantageous when the model, data flow, and software fit that design, yet its specialization does not make it a better graphics renderer or a universal replacement for CPUs and GPUs.

Inside the graphics card

- A graphics card is a complete assembly, while the GPU is the processor mounted on its printed circuit board. Around the GPU sit graphics memory chips, a connection to the computer's motherboard, and display outputs. Power circuitry converts the card's incoming electricity to the levels its components need. The resulting heat travels through a heat sink and, in the example shown by Branch Education, heat pipes and fans. These supporting parts matter: a processor capable of many calculations still needs a steady supply of data and power, and it must remain cool enough to operate at its intended speed.
- When a game loads a scene, models and textures stored on a drive ultimately move into video memory (VRAM), where the GPU can read them repeatedly while drawing frames. On-chip caches hold a smaller portion close to the computing units; memory controllers connect the chip to the much larger VRAM. The card's wide memory interface is designed to feed many calculations at once. Bandwidth describes how much data can cross that interface per second, so a scene with detailed textures or frequent data transfers can be limited by memory movement even when arithmetic units are available. The capacity of VRAM and the speed of access both affect what the card can handle.

Parallel work on the GPU

- The GPU chip contains many streaming multiprocessors (SMs), each with scheduling hardware and arithmetic units. Ordinary shader calculations use those units for operations such as multiplying two values and adding a third, a common step in vector and matrix work. An SM can keep many threads ready so that another group can run while one waits for data. Counting arithmetic units alone therefore tells us little about a finished game's speed: the work must be divided into independent pieces, supplied with inputs, and completed within each frame's time budget.
- NVIDIA's CUDA model makes this organization more precise. A program launches a grid of thread blocks; each block is assigned to an SM, and its threads are scheduled in groups called *warps*. A warp has 32 threads that execute in a single-instruction, multiple-thread (SIMT) style. They run the same program on different data, such as different vertices of a cowboy hat, while retaining their own state. The GPU schedules many warps across its SMs rather than giving every software thread a permanent arithmetic unit. This hierarchy helps the card process large collections of similar tasks without turning one vertex calculation into a long serial queue.
- Threads in a warp can take different branches, but divergence has a cost: the warp executes the needed paths with threads on other paths temporarily inactive. Rendering the same transform across many vertices is efficient because those threads usually follow similar instructions. Working data also has several homes. Registers and shared memory inside an SM are fast but limited; caches reuse data across accesses; VRAM holds far more data at a greater access cost. Keeping related inputs nearby and arranging memory reads sensibly can matter as much as adding arithmetic hardware.
- Specialized units complement the general shader arithmetic. Tensor units accelerate suitable matrix operations in neural-network workloads and some image-reconstruction techniques. Ray-tracing units help find intersections between rays and scene geometry, which can support reflections or shadows. The ordinary shader units still do much of the geometry and coloring work. In practice, a card combines these resources with its memory system; no single unit produces a complete image on its own. The locomotive scene in the next video shows how the work is divided from geometry to the displayed frame.

From a 3D scene to screen pixels

- Branch Education's game scene contains a locomotive, station, lights, and other objects represented by triangle meshes. Each vertex begins with coordinates relative to its own model. A vertex stage transforms those coordinates to place the object in the world, then into the camera's view, and finally through a perspective projection toward the screen. The camera's position, direction, and field of view determine which parts of the scene can appear. These transformations are matrix calculations applied to many vertices independently, giving the GPU the parallel work described above.
- The transformed vertices provide screen positions and depth values, as well as attributes that later stages can use for color and texture. Three vertices form a projected triangle, but that triangle has yet to become pixels. Rasterization tests where it covers the screen's sampling grid and produces *fragments*: candidate contributions to screen locations. The GPU can interpolate depth and other vertex attributes across each triangle. Thus a locomotive's broad side can occupy many screen samples even though its surface is described by a much smaller set of triangle corners.
- Visibility requires another decision. A depth, or z-, buffer records the nearest surface found at each sample. If a fragment from the locomotive is closer than the station fragment already there, its depth can replace the stored value; a farther fragment can be rejected. Depth is interpolated across a slanted triangle, so intersecting surfaces can be compared at individual screen locations rather than as whole objects. The color buffer holds the image being assembled. Together, these buffers prevent a hidden rear surface from covering the visible front merely because it was processed later.

Lighting and finishing the frame

- A triangle edge rarely matches the square pixel grid. If the renderer treats a partly covered pixel as wholly inside or outside, diagonal edges look jagged. Supersampling reduces this aliasing by evaluating several positions within each output pixel and combining their results. A pixel partly covered by the locomotive and partly by the background then takes an intermediate edge color. The improvement costs extra work, which is why games use different anti-aliasing methods depending on their performance budget.
- Fragment shading determines how a visible surface looks under light. A material or texture supplies a base color, while the surface normal describes the direction it faces. For a simple diffuse light, brightness depends on the angle between that normal and the direction of the light: for unit vectors $\mathbf{n}$ and $\mathbf{l}$, the contribution follows $\max(0, \mathbf{n}\cdot\mathbf{l})$. A face aimed toward the sun is brighter than one turned away. Ambient light can keep unlit areas visible; additional lights, highlights, and shadows add further calculations. The locomotive's dark metal therefore changes appearance as the sun and camera move, even though its underlying material remains the same.
- Using one normal for an entire triangle makes curved surfaces appear faceted. For smooth shading, the renderer can assign normals to the triangle's vertices and interpolate them across its interior. Each fragment then receives a slightly different direction for its lighting calculation, creating a gradual change in brightness across the locomotive's rounded body. Interpolation changes the apparent surface lighting, not the outline of the mesh. This distinction shows how a collection of flat triangles can suggest a smooth form without replacing its geometry.
- Some games supplement this rasterized frame with ray tracing for selected reflections, shadows, or lighting effects. Others render at a lower resolution and use an upscaling method to reconstruct a sharper output; neural techniques can use the card's tensor hardware for that task. These additions have different costs and are optional. Whenever the camera or scene changes, geometry, visibility, and shading must be updated for the next frame. The finished image emerges from parallel arithmetic, timely access to memory, and a sequence of decisions about what reaches each screen pixel.